

UPWORK PROPOSAL

ASSISTANT

Comprehensive Database System Design Document

Technical Specification Manual — Version 2.0



Project Development Group Members:

Sr. No	Group Member Name	Roll Number / ID
1	Tayyab Shehzad	NUM-BSCS-2024-77
2	Najeeba	NUM-BSCS-2024-61
3	Kashif Ali	NUM-BSCS-2024-29

Department of Computer Sciences

Namal University

Mianwali, Pakistan

Document Submission Date: 21st June, 2026

Contents

1	Project Overview & Requirement Analysis	4
1.1	Introduction	4
1.2	System Scope and Domain Rules	4
1.3	Functional Requirements	5
1.4	Non-Functional Requirements	5
2	Conceptual Database Design	6
2.1	Entity Relationship Architecture Summary	6
2.2	Data Dictionary Specifications	6
2.2.1	1. Person Table	6
2.2.2	2. Freelancer Table	7
2.2.3	3. Client Table	7
2.2.4	4. Admin Table	7
2.2.5	5. Skills Table	8
2.2.6	6. Freelancer Skill Map Table	9
2.2.7	7. Job Posting Table	9
2.2.8	8. Job Skill Map Table	10
2.2.9	9. Proposal Template Table	10
2.2.10	10. Reliability Score Table	10
2.2.11	11. Freelancer Analytics Table	10
2.2.12	12. Generated Proposals Table	11
3	Logical Database Design	16
3.1	Relational Schema Mapping Rules	16

3.2	Functional Dependencies	16
4	Implementation	17
4.1	Database Engine and Environment Settings	17
4.2	Database Programmability Objects	17
4.3	Interface Payload Mapping	17

List of Tables

2.1	Person Table Schema Definition	7
2.2	Freelancer Table Schema Definition	8
2.3	Client Table Schema Definition	8
2.4	Admin Table Schema Definition	9
2.5	Skills Table Schema Definition	9
2.6	Freelancer Skill Map Table Schema Definition	10
2.7	Job Posting Table Schema Definition	11
2.8	Job Skill Map Table Schema Definition	12
2.9	Proposal Template Table Schema Definition	12
2.10	Reliability Score Table Schema Definition	13
2.11	Freelancer Analytics Table Schema Definition	14
2.12	Generated Proposals Table Schema Definition	15
4.1	Database Programmability Objects Directory	18

Chapter 1

Project Overview & Requirement Analysis

1.1 Introduction

The modern freelance economy has experienced exponential growth, leading to intense competition within digital marketplaces like Upwork. Winning service contracts relies heavily on a freelancer's ability to craft a highly customized, technically accurate, and rapid response to open client briefs.

The **Upwork Proposal Assistant** is engineered to address this bottleneck. By providing a relational infrastructure that evaluates historical client habits, ranks technical skill matrices, catalogs dynamic boilerplate text templates, and logs proposal outcome tracking data, the system serves as an automation engine that accelerates the client-freelancer matching pipeline.

1.2 System Scope and Domain Rules

The operational architecture handles detailed account segmentation and transactional logging rules:

- **User Management and Profile Separation:** The system tracks base credentials via a generalized parent classification, which branches out into dedicated, context-specific behaviors for freelancers (handling rates and experience), clients (handling tracking matrices), and internal monitoring administrators.
- **Marketplace Interconnectivity:** Clients deploy specific job listings requiring targeted technical proficiencies. The system maintains bridge mapping entities that balance many-to-many relationship mappings without duplicating data.

- **Automation and Risk Analysis Evaluation:** To protect freelancers from bad-faith actors or low-conversion biddings, the backend analyzes past metrics to compute reliability balances, tracking risk classifications via automated triggers.

1.3 Functional Requirements

1. **FR-1 (Profile Administration):** Enforce strict email and portfolio verification checks during identity creation.
2. **FR-2 (Listing Ingestion):** Enable clients to post detailed projects bounded by experience level limits and designated maximum budget thresholds.
3. **FR-3 (Template Curation):** Provide administrators with interface avenues to manage boilerplate variants while logging continuous usage histories.
4. **FR-4 (Analytical Automation):** Automatically trigger risk and reliability updating metrics whenever historical tracking records change.

1.4 Non-Functional Requirements

1. **NFR-1 (Data Integrity):** Maintain complete referential constraint integrity across all multi-table relationship updates using cascading operational policies.
2. **NFR-2 (Scalability):** Structure physical schemas in Third Normal Form (3NF) to minimize transactional storage anomalies during heavy payload execution.
3. **NFR-3 (Performance Auditing):** Implement pre-compiled database procedures to handle heavy processing routines without introducing high connection lag.

Chapter 2

Conceptual Database Design

2.1 Entity Relationship Architecture Summary

The core configuration maps operational variables cleanly to minimize structural meta-data flaws. The base schema architecture isolates general personal identity descriptors into a master index table, allowing specialised downstream extensions to inherit operational properties cleanly.

The system tracks dynamic connections across client listings, specific skills, template content layouts, and historical analytical metrics logs. This systematic grouping guarantees that any input configuration is cleanly tracked, helping avoid common data anomalies when processing simultaneous updates.

2.2 Data Dictionary Specifications

The physical domain fields, storage allocations, primary keys (PK), foreign keys (FK), and structural constraint configurations for all 12 relations constituting the system schema are systematically cataloged within the grid structures below.

Each table is accompanied by a technical description detailing its design rationale, normalization boundaries, and business rule configurations:

2.2.1 1. Person Table

The **Person** table acts as the base entity for user identification across the system. It abstracts shared security credentials, registration locations, and communication fields to prevent duplicate records across different user levels.

Table 2.1: Person Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
person_id	INT	PK	NOT NULL	System-generated auto-increment sequence key for unique row tracking.
f_name	VARCHAR(50)	—	NOT NULL	User's first name; validated to handle alpha character input configurations.
Lname	VARCHAR(50)	—	NOT NULL	User's last name; provides mandatory descriptive structural labeling text.
email	VARCHAR(100)	—	UNIQUE	Primary communications handle bounded by regular expression validation controls.
country	VARCHAR(50)	—	NULL	Optional geographical string logging the country location of the user.
upwork_profile_url	VARCHAR(255)	—	UNIQUE	Verified external link to token checking the user's active portfolio profile.

2.2.2 2. Freelancer Table

The **Freelancer** entity extends the core **Person** table via a 1:1 relationship layout. It isolates specific contractor metrics like billing tier weights and total experience tenure variables, isolating them from general platform accounts.

2.2.3 3. Client Table

The **Client** table structures account metadata for hiring managers. It tracks aggregate success parameters and history fields, isolating corporate profile structures away from freelancer configurations.

2.2.4 4. Admin Table

The **Admin** structure handles operational metrics for platform moderators. It isolates internal system functions from the public marketplace, defining roles for template oversight and profile verification workflows.

Table 2.2: Freelancer Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>freelancer_id</code>	INT	PK, FK	NOT NULL	Unique key mapping 1:1 with the parent Person entity; supports cascading operational paths.
<code>password</code>	VARCHAR(255)	—	NOT NULL	Secure, cryptographically hashed authentication phrase string.
<code>hourly_rate</code>	DECIMAL(6,2)	—	NOT NULL	Professional hourly service billing rate, restricted by check constraints to values ≥ 0.00 .
<code>experience_years</code>	INT	—	NULL	Optional integer tracking verified history within the technical marketplace.

Table 2.3: Client Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>client_id</code>	INT	PK, FK	NOT NULL	Unique key tracking individual corporate managers; maps directly back to the base Person row.
<code>company_name</code>	VARCHAR(100)	—	NULL	Registered enterprise business name for professional corporate accounts.
<code>success_rate</code>	DECIMAL(5,2)	—	NULL	Historical hiring score balance tracing successful project closures.
<code>feedback</code>	TEXT	—	NULL	Open description field storing qualitative review logs collected over time.

2.2.5 5. Skills Table

The **Skills** master lookup index catalogs the core technical keywords used across the platform. It prevents text inconsistencies by standardizing entry names for freelancers and project listings.

Table 2.4: Admin Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>admin_id</code>	INT	PK, FK	NOT NULL	Inherits baseline personal attributes from Person , granting administrative access rights.
<code>role</code>	VARCHAR(50)	—	NOT NULL	Functional assignment label defining system duties (e.g., 'Template Manager').
<code>joined_date</code>	DATE	—	NOT NULL	Calendar date marking when administrative permissions were authorized.
<code>status</code>	VARCHAR(20)	—	NOT NULL	Operational status tracking state toggle, restricted to 'Active', 'Suspended', or 'Inactive'.

Table 2.5: Skills Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>skill_id</code>	INT	PK	NOT NULL	Primary tracking surrogate identifier managing discrete knowledge profile assets.
<code>skill_name</code>	VARCHAR(100)	—	UNIQUE	Distinct standardized skill label text string (e.g., 'MySQL Engine', 'LaTeX Typography').
<code>skill_level</code>	VARCHAR(30)	—	NOT NULL	Structural mapping bounds restricted exclusively to 'Beginner', 'Intermediate', or 'Expert' states.

2.2.6 6. Freelancer Skill Map Table

The `Freelancer_Skill_Map` is an associative entity that breaks down a many-to-many relationship layout. It maps individual contractor profiles directly to their verified technical competencies without duplicating text strings.

2.2.7 7. Job Posting Table

The `Job_Posting` table logs open client briefs uploaded to the marketplace. It manages essential metadata, including required experience limits, project lengths, and tracking

Table 2.6: Freelancer Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>freelancer_id</code>	INT	PK, FK	NOT NULL	Composite primary key component; links to the targeted active profile inside Freelancer .
<code>skill_id</code>	INT	PK, FK	NOT NULL	Composite primary key component; links to the targeted competency attribute inside Skills .

variables for visibility rules.

2.2.8 8. Job Skill Map Table

The `Job_Skill_Map` is an associative entity that maps listings to their technical prerequisites. This bridging structure enables the system to calculate matching percentages for incoming applications.

2.2.9 9. Proposal Template Table

The `Proposal_Template` table catalogs reusable text modules managed by administrators. It monitors performance metrics to help freelancers identify layouts with high conversion rates.

2.2.10 10. Reliability Score Table

The `Reliability_Score` entity logs client risk history data. It tracks analytical summaries generated by backend procedures to help identify reliable accounts before application submission.

2.2.11 11. Freelancer Analytics Table

The `Freelancer_Analytics` table tracks historical application performance. It maintains running aggregates of total, accepted, and rejected applications to feed statistical modeling routines.

Table 2.7: Job Posting Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>job_id</code>	INT	PK	NOT NULL	Primary index identifier tracking distinct contract listings.
<code>client_id</code>	INT	FK	NOT NULL	Referential connection mapping the listing back to its original creator inside the <code>Client</code> table.
<code>job_title</code>	VARCHAR(150)	—	NOT NULL	Descriptive headline summarizing the core engineering request details.
<code>job_description</code>	TEXT	—	NOT NULL	Comprehensive documentation text outlining the full scope parameters of the contract.
<code>experience_level</code>	VARCHAR(20)	—	NOT NULL	Enforces explicit boundary check rules: 'Entry', 'Intermediate', or 'Expert'.
<code>job_type</code>	VARCHAR(20)	—	NOT NULL	Financial structure flag selector; enforces options between 'Fixed-Price' or 'Hourly'.
<code>project_duration</code>	VARCHAR(50)	—	NULL	Dynamic text string estimating the expected timeline window to reach milestones.
<code>budget_amount</code>	DECIMAL(10,2)	—	NOT NULL	Financial funding allocated for development tasks; verified via logic checks to remain > 0.00 .
<code>posted_date</code>	TIMESTAMP	—	NOT NULL	Automatic system timestamp logging the deployment instance of the contract listing.
<code>proposal_count</code>	INT	—	DEFAULT 0	Active counter tracking the number of incoming bids submitted against this project listing.

2.2.12 12. Generated Proposals Table

The `Generated Proposals` table records transactional logging for compiled cover letters. It connects contractors, target clients, and reference templates, maintaining complete audit trails for every submission.

Table 2.8: Job Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>job_id</code>	INT	PK, FK	NOT NULL	Composite link mapping the requirement to a project listing in the <code>Job.Posting</code> table.
<code>skill_id</code>	INT	PK, FK	NOT NULL	Composite link mapping the requirement to a skill profile entry in the <code>Skills</code> table.

Table 2.9: Proposal Template Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>template_id</code>	INT	PK	NOT NULL	Unique surrogate key managing individual structural boilerplate layout parameters.
<code>admin_id</code>	INT	FK	NULL	Relational reference tracing authorization back to a team member in the <code>Admin</code> table.
<code>template_name</code>	VARCHAR(100)	—	NOT NULL	Descriptive name label for rapid layout selection (e.g., 'Backend Deployment Layout').
<code>category</code>	VARCHAR(50)	—	NOT NULL	Business domain classification string used for rapid indexing filter systems.
<code>template_content</code>	TEXT	—	NOT NULL	Raw text boilerplates utilizing dynamic variables parsed during assembly procedures.
<code>usage_count</code>	INT	—	DEFAULT 0	Active tracking tally tracing how many times an asset was used by contractors.
<code>success_rate</code>	DECIMAL(5,2)	—	NULL	Statistical performance conversion ratio rating computed dynamically over time.

Table 2.10: Reliability Score Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
score_id	INT	PK	NOT NULL	Sequential unique surrogate record identity token tracking risk analytical historical loops.
client_id	INT	FK	NOT NULL	Relational link routing analytics back to the target client inside the Client table.
score	DECIMAL(5,2)	—	NOT NULL	Numerical score index indicating evaluated behavioral consistency values.
risk_level	VARCHAR(20)	—	NOT NULL	Categorical evaluation classification bound by strict checks to 'Low', 'Medium', or 'High'.
recommendation_level	VARCHAR(50)	—	NOT NULL	Action guide text indicating computed choices like 'Highly Recommended' or 'Avoid Account'.
calculated_date	TIMESTAMP	—	NOT NULL	System execution timestamp logging when the analysis background thread refreshed this record.

Table 2.11: Freelancer Analytics Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
<code>analytics_id</code>	INT	PK	NOT NULL	Surrogate identity row token tracking active contractor metrics snapshots.
<code>freelancer_id</code>	INT	FK	NOT NULL	Relational link routing metrics back to the corresponding profile in the Freelancer table.
<code>total_proposals</code>	INT	—	DEFAULT 0	Continuous system tally tracking the total count of all submitted application attempts.
<code>accepted_proposals</code>	INT	—	DEFAULT 0	Tally record tracking successful client contract hires finalized across the platform.
<code>rejected_proposals</code>	INT	—	DEFAULT 0	Tally record tracking archived or passed applications rejected across contract listings.
<code>success_rate</code>	DECIMAL(5,2)	—	DEFAULT 0.00	Dynamic mathematical calculation percentage storing proposal conversion ratios.

Table 2.12: Generated Proposals Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
proposal_id	INT	PK	NOT NULL	Unique index identifier tracking individual cover letter generation instances.
freelancer_id	INT	FK	NOT NULL	References Freelancer ; identifies the structural author producing the bid documentation.
client_id	INT	FK	NOT NULL	References Client ; links the payload to the targeting hiring profile manager.
job_id	INT	FK	NOT NULL	References Job_Posting ; anchors the bidding event to an active open marketplace listing.
template_id	INT	FK	NOT NULL	References Proposal_Template ; catalogs the specific blueprint configuration asset used.
proposal_content	TEXT	—	NOT NULL	Final structural letter layout text generated by parsing models for client transmission.
generated_date	TIMESTAMP	—	NOT NULL	Tracks the calendar system time when the generative parsing automation engine compiled the row.
proposal_status	VARCHAR(30)	—	NOT NULL	State tracker bound by check options to 'Draft', 'Submitted', or 'Accepted'.
submission_date	TIMESTAMP	—	NULL	Tracks the exact moment when files were released out into the marketplace interfaces.

Chapter 3

Logical Database Design

3.1 Relational Schema Mapping Rules

The conceptual layout maps to normalized logical structures through precise rules. The 1:1 specialization line linking **Person** to sub-entities uses matching shared primary keys, preventing blank fields and avoiding wide sparse tables. Many-to-many relationships are resolved using intermediate mapping tables with composite keys, keeping data footprints small.

3.2 Functional Dependencies

All data fields map directly and explicitly to primary candidate keys to fulfill Third Normal Form (3NF) requirements:

- $\text{person_id} \rightarrow \{\text{f_name}, \text{Lname}, \text{email}, \text{country}, \text{upwork_profile_url}\}$
- $\text{freelancer_id} \rightarrow \{\text{password}, \text{hourly_rate}, \text{experience_years}\}$
- $\text{job_id} \rightarrow \{\text{client_id}, \text{job_title}, \text{job_description}, \text{experience_level}, \text{job_type}, \text{budget_amount}, \text{posted_date}, \text{proposal_count}\}$
- $\text{analytics_id} \rightarrow \{\text{freelancer_id}, \text{total_proposals}, \text{accepted_proposals}, \text{rejected_proposal_success_rate}\}$

Since every non-key column depends solely on the full primary key, the schema design is completely free of partial or transitive dependencies.

Chapter 4

Implementation

4.1 Database Engine and Environment Settings

The physical relational architecture is implemented on a local deployment of the MySQL 8.0 Engine using the InnoDB storage engine. InnoDB ensures complete ACID compliance, managing transaction locks safely during concurrent update procedures. The global database character collation utilizes `utf8mb4_unicode_ci` to support secure URL logging strings without truncation faults.

4.2 Database Programmability Objects

To automate performance tracking and reduce network overhead, database routines and triggers handle core calculation workflows directly within the storage engine:

4.3 Interface Payload Mapping

The application interface connects securely via decoupled connection drivers. Transaction blocks wrap all proposal operations, ensuring that whenever a letter record updates, lookup metrics sync simultaneously or roll back cleanly if an error occurs.

Table 4.1: Database Programmability Objects Directory

Sr.	Object Name	Object Type	Description	Input Params	Output / Return
01	Calculate-Reliability	Stored Procedure	Queries historical contract closing distributions to evaluate client profiles, calculate risk metrics, and append structured summaries to active score records.	c_id INT, raw_score DECIMAL	Inserts a calculated evaluation record into the Reliability_Score table.
02	Increment-TemplateUsage	Trigger	Automation rule listening on the submission layer that captures new records and runs atomic counts to sync template metrics.	System Event (AFTER INSERT)	Increments real-time performance balances in the Proposal_Template table.
03	SyncFreelancer-Metrics	Stored Procedure	Audits performance records across the database, recalculating application totals, rejection sums, and success percentages.	f_id INT	Commits updated calculations directly to the Freelancer_Analytics table.

Bibliography

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [2] MySQL Reference Manual, “Stored Procedures, Functions, and Triggers Infrastructure,” Oracle Corp, 2026.

Appendix: AI Usage and Prompts

In compliance with project design evaluation guidelines, the development prompts used with generative artificial intelligence models are documented below:

- **Prompt 1 (Schema Refactoring):** “Review my Upwork database design layout attributes. Fix any potential decimal data type truncation bottlenecks within the job posting financial budgets and client reliability performance score range values to ensure it safely handles up to 100% metric values without runtime crashes.”
- **Prompt 2 (Transactional Scaling):** “Generate a fully expanded, interconnected relational seed script mapping out 2 distinct administrative roles (one for system user metrics analysis, one for outgoing template tracking) along with more than 20 rows of realistic transactional mock records per operational table to test structural performance.”
- **Prompt 3 (Documentation Mapping):** “Map my updated relational set structures into a comprehensive logical validation report structured under formal Chapter 3 and Chapter 4 sections following Namal University project templates.”